Sticky Jump Diffusions for Adaptive Discrete Generation and Reasoning

Pascal Jutras-Dubé
Purdue University
Department of Computer Science
West Lafayette, Indiana, United States of America

Patrick Pynadath
Purdue University
Department of Computer Science
West Lafayette, Indiana, United States of America

Jeremy Lu
Purdue University
Department of Computer Science
West Lafayette, Indiana, United States of America

Yuan Gao
Purdue University
Department of Mathematics
West Lafayette, Indiana, United States of America

Ruqi Zhang
Purdue University
Department of Computer Science
West Lafayette, Indiana, United States of America

Abstract

Diffusion models have been employed to generate discrete (e.g. text, molecules, and tabular data) or continuous data samples (e.g. images, audio, and robot trajectories) separately. We develop a principled framework that unifies hybrid discrete/continuous diffusions and learns data-adaptive unmasking schedules. Specifically, we formalize *Sticky Jump Diffusions* (SJD), an Itô diffusion that diffuses in continuous space and sticks to anchors in discrete space with first-time events taken from an inhomogeneous Poisson point process. We compute the reverse-time hazard, determining when and where to commit, without estimating intractable time-marginals. Together with a cross-entropy allocator and classifier-parametrized score, we demonstrate a splitting scheme: ancestral sampling in the reverse process involves a continuous step from the score and a discrete commitment step from the classifier.

1 Introduction

With the growth of Diffusion Large Language Models (dLLMs), there has been interest in extending hybrid diffusion frameworks to sample both discrete and continuous modalities simultaneously. Masked diffusion models (MDMs) have emerged as alternatives to autoregressive models for efficient, high-quality generation of discrete data including text, images, and molecules. One crucial component is the unmasking schedule that determines the location and timing of commitments to discrete symbols [1, 5]. Most existing schedules are time-only, which controls the expected fraction of masked positions, resulting in uniformly-random unmasking orders. Recent work has shown that models tend to train on non-uniformly "hard" masking subproblems, leading to the phenomenon that the ordering of revealed tokens can heavily impact the trajectory of generated samples [4]. Current hybrid approaches unifying discrete and continuous diffusion rely on inference-time heuristics and/or time-only schedules to improve sample quality [6, 8, 9]. However, there has not yet been an effective hybrid framework encoding this information in the probabilistic model itself, which would otherwise sidestep the need to hand-craft explicit ordering heuristics.

We show that the problem of token-adaptive unmask ordering can be solved as a direct consequence of a hybrid discrete-continuous framework through the lens of jump diffusions: diffusions that jump off an anchor state at the first-time arrival and diffuse in the forward process towards Gaussian-distributed noise in continuous space (e.g. $\mathbb{R}^d$). Derived from jump diffusion theory, the reverse-time process can be split into a continuous reversal step in the stochastic differential equation (SDE) and a jump kernel that sticks back onto discrete space [2].

The computed time-reversal hazard provides a signal of commitment timing and location, while conditioning on the forward state in continuous space. Therefore, during inference, state-dependent unmasking is analogous to sampling from the jump kernel by the total hazard intensity. As such, token-adaptive ordering is already governed by the probabilistic model. Regressing over the plug-in hazard in L^2 space leads to the Bayes-optimal hazard, which can be used to compute

the denoising posterior $p_t(a \mid y)_{a\in\mathcal{A}}$. In our experiments, this posterior is approximated by the classifier $P_\theta(a \mid Y, t)$.

To our knowledge, this is the first work bridging Jump Diffusions with modeling discrete and continuous distributions, and connecting the SJD framework to adaptive token ordering and unmasking schemes.

2 Sticky Jump-Diffusion (SJD) Models

We provide formalized definitions for the SJD framework.

Let the state space be $\mathsf{X} = \mathbb{R}^d$ and the anchor set be $\mathcal{A} = \{a_i\}_{i=1}^n \subset \mathbb{R}^d$. The continuous space that we are interested in is $\mathsf{X}_\mathcal{A} = \mathsf{X} \setminus \mathcal{A}$ over the chosen set of anchors. Let p_t be absolutely continuous on this space and atoms on $\mathcal{A}$.

Let $T \in \mathbb{R}$ be the horizon of the Markov process.

2.1 Forward-time Dynamics

For the continuous region $\mathsf{X}_\mathcal{A}$, we write the forward SDE as:

$$\mathrm{d}\vec{X}_t = \vec{b}_t(\vec{X}_t)\mathrm{d}t + g_t \mathrm{d}\vec{W}_t,$$

where b_t denotes the drift term parameterized by the current state X_t, and $g_t > 0$ is the diffusion coefficient scaled by the standard Brownian motion increment $\mathrm{d}\vec{W}_t$.

For the discrete region, as long as $\vec{X}_t \in \mathsf{X}_\mathcal{A}$,

$$Y \sim R_a(\cdot),$$

according to the first-time arrival event dependent on the hazard $\vec{\lambda}_t$. We denote $R_a(\mathrm{d}y)$ to be the unsticking kernel over anchors and $r_a(y) = R_a(\mathrm{d}y)$ to be one such distribution over a particular anchor $a \in \mathcal{A}$.

The forward process may *not* jump into anchors. We assume a formulation with one initial time-event, following the design of a Poisson point process.

In our experiments, we apply the Variance Preserving (VP)-matched SDE, according to [7],

$$\mathrm{d}X_t = -\frac{1}{2}\beta(t)\vec{X}_t\mathrm{d}t + \sqrt{\beta(t)\left(1 - \exp\left(-2\int_0^t \beta_s \mathrm{d}s\right)\right)}\mathrm{d}\vec{W}_t.$$

The closed form VP perturbation kernel is written as $\vec{X}_t \mid \vec{X}_0 \sim \mathcal{N}(\alpha_t x_0, \sigma_t^2 \mathbf{I})$, where $\alpha_t := \exp(-1/2\int_0^t \beta_s ds)$ and $\sigma_t^2 := 1 - \exp(-\int_0^t \beta_s ds)$. We set $t \in [\epsilon, 1]$ due to the vanishing variance with $\epsilon > 0$.

Under the VP-SDE, the marginal $q_t(\cdot \mid x_0)$ converges weakly to $\mathcal{N}(0, \mathbf{I})$ as $t \to T$, for any $x_0 \in \mathcal{A}$.

2.2 Reverse-time Dynamics

For $x \in \mathsf{X}$, we write the reverse-time evolution as an SDE,

$$\mathrm{d}\overleftarrow{X}_\tau = \overleftarrow{b}_t(\overleftarrow{X}_\tau)\mathrm{d}t + g_t \mathrm{d}\overleftarrow{W}_\tau,$$

with $\tau = T - t$. We set $\overleftarrow{b}(x) = \overleftarrow{b}_t - g_t^2 \nabla \log p_t(x)$ for our experiments where $\nabla \log p_t(x)$ is the score of the perturbed distribution.

The sticky jump law has per-anchor backward intensities

$$\begin{aligned}\Lambda_t^\star(a \mid y) &\equiv \overleftarrow{J}_t(y, \{a\}) \\ &= \frac{m_t(a)}{p_t(y)}\vec{\lambda}_t r_a(y), \quad y \in \mathsf{X}_\mathcal{A},\ a \in \mathcal{A},\end{aligned} \tag{1}$$

Definition 2.1. The total intensity at $y \in \mathsf{X}_\mathcal{A}$ is

$$\overleftarrow{\lambda}_t^\star(y) := \sum_{a\in\mathcal{A}} \Lambda_t^\star(a \mid y) = \frac{\vec{\lambda}_t \sum_{a\in\mathcal{A}} m_t(a) r_a(y)}{p_t(y)}. \tag{2}$$

The relative allocation across anchors at (y, t) is

$$\pi_t^\star(a \mid y) := \frac{\Lambda_t^\star(a \mid y)}{\sum_{b\in\mathcal{A}} \Lambda_t^\star(b \mid y)} = \frac{m_t(a) r_a(y)}{\sum_{b\in\mathcal{A}} m_t(b) r_b(y)}, \tag{3}$$

3 Loss Functions and Parametrized Distributions

We outline the loss functions to train the neural networks that parametrize the posterior distributions of interest.

The total intensity can be parameterized by the neural network parameter vector θ:

$$\Lambda_\theta(a \mid y, t) := \lambda_\theta(y, t) P_\theta(a \mid y, t), \qquad \sum_{a\in\mathcal{A}} P_\theta(a \mid y, t) = 1, \tag{4}$$

where $\lambda_\theta(y, t)$ is our state-dependent hazard and $P_\theta(a \mid y, t)$ is the classifier.

Notably, we may compute the score of the forward process's Gaussian, assuming the forward kernel is VP,

$$\begin{aligned}\nabla_y \log p_t(y) &= \sum_{a\in\mathcal{A}} p_t(a \mid y) \nabla_y \log q_t(y \mid a) \\ &= -\frac{y - \alpha(t)\mu_t(y)}{\sigma_t^2},\end{aligned}$$

most commonly known as Tweedie's formula [3]. $\mu_t(y) = \mathbb{E}[a \mid y] \approx \mu_\theta(y, t) = \sum_{a\in\mathcal{A}} P_\theta(a \mid y, t) a$, since in our formulation $P_\theta(a \mid y, t)$ generates the Monte-Carlo (MC) estimate in place of $p_t(a \mid y)$.

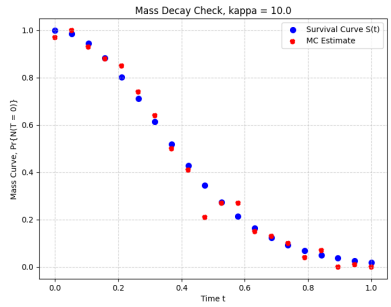

Figure 1. Validation of the survival curve $S(t)$ against an MC estimate of $P(N(T) = 0)$ with $\kappa = 10$. The analytic curve and empirical fraction of uncommitted sites confirms that the hazard schedule and inverse-CDF sampler are correctly implemented.

For brevity, we omit proofs in this report for the Rao-Blackwellized plug-in hazard

$$\lambda_{\text{plug},\theta}(y,t) = \sum_{a\in\mathcal{A}} \frac{\vec{\lambda}_t(a)S_a(t)r_a(y)}{q_t(y \mid a)} P_\theta(a \mid y,t),$$

without further showing that it is Bayes optimal.

4 Classifier Cross-Entropy

Further, we define a loss function to obtain the classifier distribution outlined in the previous section for VP score estimation. Let $t \in [0,T]$ be the unsticking time for the anchor $a \in \mathcal{A}$, and $Y \sim r_a(\cdot)$ be the location after the jump. Given that $\vec{D} = \{(t,a,Y)\}$ is the tuple for the forward unstick event,

$$\mathcal{L}_{\text{cls}} = \sum_{(t,a,Y)\in\vec{D}} -\log P_\theta(a \mid Y,t). \tag{5}$$

The computation of the cross entropy $-\log P_\theta$ implicitly lowers the probability of unseen anchor events, which are masked out in practice, and maximizes the likelihood of observing the anchor a at position y.

5 Denoising Hazard Matching (DHM)

Let

$$m_t(a) = p_0(a)S_a(t), \quad S_a(t) = \exp\left\{-\int_0^t \vec{\lambda}_s(a)\mathrm{d}s\right\}.$$

be the anchor mass distribution and survival curve for the first-time arrival, respectively.

Assuming $q_t(y \mid a) > 0$,

$$\widehat{\lambda}_t(y,x_0) := \frac{\vec{\lambda}_t(x_0)S_{x_0}(t)r_{x_0}(y)}{q_t(y \mid x_0)}. \tag{6}$$

Intuitively, $\frac{r_{x_0}(y)}{q_t(y|x_0)}$ provides the proximity ratio between the unsticking and perturbed locations. For example, if the variance of $r_{x_0}(y)$ is high (i.e., r_{x_0} is permissive of y's jump location) while that of $q_t(y \mid x_0)$ is low, the hazard $\widehat{\lambda}_t(y,x_0)$ encourages a sooner jump arrival than expected.

Theorem 5.1 (Denoising Hazard Matching)**.** *Let $t \in [\epsilon, T]$ as stated before and $w(t) \geq 0$ be a time-dependent weight. Let $X_0 \sim p_0(\cdot)$ distributed from the prior distribution and $X_t \sim q_t(\cdot \mid X_0)$ distributed as the forward noised sample from the perturbation kernel. Consider the DHM objective,*

$$\mathcal{L}_{\text{DHM}}(\theta) := \mathbb{E}\left[w(t)\left(\lambda_\theta(X_t,t) - \widehat{\lambda}_t(X_t,X_0)\right)^2\right], \tag{7}$$

Proof. Omitted. □

It naturally follows that

$$\lambda^*_\theta(y,t) = \operatorname*{argmin}_\theta \mathcal{L}_{\text{DHM}}(\theta),$$

which is a generalization of time-dependent schedules (cosine, linear, etc.). Specifically, $\lambda^*_\theta(t)$ measures the jump intensity at time t, and conditioning it further on $y \in \mathrm{X}_{\mathcal{A}}$ enables state-dependent token ordering during inference.

6 Results

6.1 Training Dynamics

We trained the Sticky Jump Diffusion (SJD) classifier on a point dataset in $\mathbb{R}^2$ with $|\mathcal{A}| = 100$ anchors arranged in two concentric moon-shaped patterns. The classifier network consists of an MLP with time embedding, consisting of 2048 hidden units with SiLU activations and dropout ($p = 0.1$) for regularization. We optimize using Adam with learning rate $\eta = 3.0 \times 10^{-3}$ over 1000 epochs with batch size 256 and $\kappa = 1.0$ (hazard scale parameter). The training/validation split is 80/10 (with 10% held-out test).

6.2 Masked vs. Unmasked Accuracy

The loss function in Eq. (5) optimizes the *masked* cross-entropy, where accuracy is computed only over samples that have not yet committed to an anchor (non-committed samples). This design follows the SJD framework: during the forward process, samples that have already unstuck and moved into continuous space should not be penalized for misclassifying their origin anchor.

Table 1. Training metrics with 1000 epochs.

Metric	Train	Validation
Loss	4.87 ± 0.08	13.0 ± 0.27
Accuracy (masked)	0.02 ± 0.0083	–
Accuracy (unmasked)	0.02 ± 0.0076	–
Commit Rate	0.20 ± 0.025	0.21 ± 0.063
Non-Commit Rate	0.80 ± 0.025	0.79 ± 0.063

6.2.1 Training Performance. The model shows consistent but modest improvement on the training set:

- **Training Loss**: ≈ 4.87 nats in later epochs
- **Validation Loss**: ≈ 13.0 nats (2.7× training loss)

The gap between masked and unmasked accuracy demonstrates that the model is learning weak signal on non-committed samples (per our loss design), while unmasked accuracy reveals that random guessing should yield $1/100 = 1\%$ performance on this classification task.

6.3 Reverse Process Visualizations

We ablate over the aforementioned splitting scheme:

- **Euler-Maruyama (EM) Step**: Apply the reverse SDE in Section 2.2.
- **Commitment Step**: Anchor indices are drawn from a categorical distribution using the classifier's predictions, and anchor samples are taken from these indices.

Figure 2 visualizes the final sample distributions. The full reverse process produces samples clustered heavily near anchors. The EM-only process reveals a more diffuse distribution shaped by the classifier's score estimate, demonstrating that the learned score function is steering samples without forcing hard commitments.

7 Future Plans

As mentioned, the classifier-induced score does not produce the desired terminal distribution of $\mathbb{R}^2$ points.

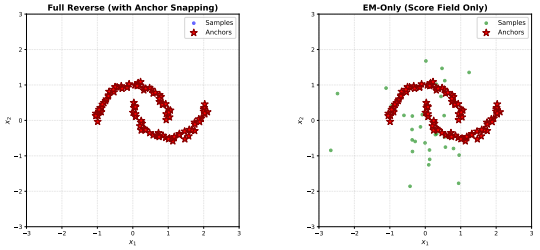

Figure 2. Comparison of full reverse process (left, classifier anchor predictions) versus EM-only (right, classifier-induced score). The full process concentrates mass near anchors due to the jumping mechanism; the EM-only process fails due to an absence of anchor commitments.

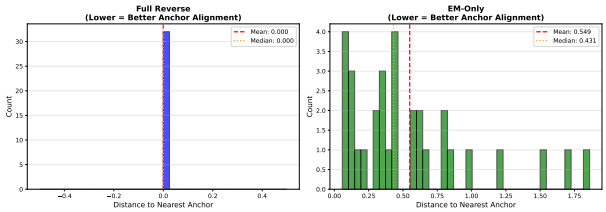

Figure 3. Histogram of minimum distance from each sample to its nearest anchor. The full reverse process achieves lower mean distance ($\mu \approx 0.3$–0.4) due to anchor snapping; the EM-only process has higher mean distance ($\mu \approx 1.0$–1.2), reflecting pure diffusion.

This may be due to numerical instability or a weak classifier network. We plan to implement an embedding table for the anchors in $\mathcal{A}$ space to allow the continuous SDE to operate in latent space.

In addition, we will test the robustness of state-dependence across different data modalities. Concretely, employing SJD for Sudoku puzzles, parallel decoding benchmarks, and molecular platforms would demonstrate that our current framework handles complex state-conditioning based on the classifier and hazard networks, rather than converging to time-only schedules. Furthermore, an anchor table will be implemented to handle text tokens.

8 Related Work

Discrete Diffusion Models. Masked diffusion models [1, 5] define forward processes over discrete token vocabularies via absorbing or uniform noise. MDLM and related work extend these to language modeling at scale. Our work differs in that the unmasking schedule is state-dependent rather than time-only.

Continuous Diffusion Models. Score-based generative models [7] define continuous diffusions via SDEs and learn to reverse them via score matching. We adopt the VP-SDE as our continuous component and use Tweedie's formula to connect the classifier posterior to the score.

Hybrid Discrete-Continuous Diffusion. MD4 [6] and RDD [8] unify discrete and continuous modalities but rely on inference-time heuristics for unmasking order. Some work [9] explore non-uniform orderings but do not encode them probabilistically. SJD derives adaptive ordering directly from the reverse-time hazard.

Jump Processes and Point Processes. Conforti and Léonard [2] establish time-reversal theory for jump diffusions, which we build on to derive the reverse sticky jump law. Inhomogeneous Poisson point processes have been used in event modeling but not, to our knowledge, in generative modeling for discrete token commitment.

9 Conclusion

We introduced Sticky Jump Diffusions (SJD), a principled hybrid discrete-continuous diffusion framework that derives state-dependent token commitment directly from the reverse-time hazard of an inhomogeneous Poisson point process. Unlike prior work relying on inference-time heuristics or time-only unmasking schedules, SJD encodes adaptive ordering probabilistically through the jump kernel. We derived the reverse-time per-anchor intensities, the plug-in hazard, and the classifier cross-entropy loss, and demonstrated a splitting scheme separating continuous EM steps from discrete commitment steps. We validate preliminary experiments on a $\mathbb{R}^2$ points dataset, validate the survival curve implementation, and confirm that the commitment mechanism produce anchor-concentrated samples. In future work, we will scale SJD to text tokens, Sudoku, and parallel decoding benchmarks to evaluate state-dependent ordering against time-only baselines.

10 Acknowledgments

The author thanks Pascal Jutras-Dubé, Patrick Pynadath, Yuan Gao, and Ruqi Zhang for their collaboration and guidance on this project.

11 Authors and Affiliations

Jeremy Lu is the main author for this report.

References

[1] Jacob Austin, Daniel D Johnson, Jonathan Ho, Daniel Tarlow, and Rianne Van Den Berg. 2021. Structured denoising diffusion models in discrete state-spaces. In *Advances in Neural Information Processing Systems*.

[2] Giovanni Conforti and Christian Léonard. 2022. Time Reversal of Markov Processes with Jumps under a Finite Entropy Condition. *Stochastic Processes and their Applications* (2022).

[3] Bradley Efron. 2011. Tweedie's Formula and Selection Bias. *J. Amer. Statist. Assoc.* 106, 496 (2011), 1602–1614.

[4] Jaeyeon Kim, Kulin Shah, Vasilis Kontonis, Sham Kakade, and Sitan Chen. 2025. Train for the Worst, Plan for the Best: Understanding Token Ordering in Masked Diffusions. In *International Conference on Machine Learning*.

[5] Aaron Lou, Chenlin Meng, and Stefano Ermon. 2024. Discrete Diffusion Modeling by Estimating the Ratios of the Data Distribution. In *International Conference on Machine Learning*.

[6] Jiaxin Shi, Kehang Han, Zhe Wang, Arnaud Doucet, and Michalis K. Titsias. 2024. Simplified and Generalized Masked Diffusion for Discrete Data. In *Advances in Neural Information Processing Systems*.

[7] Yang Song, Jascha Sohl-Dickstein, Diederik P Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. 2021. Score-Based Generative Modeling through Stochastic Differential Equations. In *International Conference on Learning Representations*.

[8] Xinyun Wang, Min Zhang, Sen Cui, Zhikang Chen, Bo Jiang, Kun Kuang, and Mingbao Lin. 2026. Reversible Diffusion Decoding for Diffusion Language Models. *arXiv preprint arXiv:2602.00150* (2026).

[9] Longxuan Yu, Yu Fu, Shaorong Zhang, Hui Liu, Greg Ver Steeg, Yue Dong, et al. 2026. Thinking Out of Order: When Output Order Stops Reflecting Reasoning Order in Diffusion Language Models. *arXiv preprint arXiv:2601.22035* (2026).